

AI技术调研与场景分析

提交人：唐立峰

学号：2023141461045

课程：软件过程与管理

项目名称：面向网课的智能监督系统

一、项目背景

本次作业是在第一次《软件过程剪裁与定义》作业的基础上继续完成的。第一次作业中，我结合课程小组项目“面向网课的智能监督系统”，对项目的软件过程进行了定义和剪裁。该项目主要面向线上课程场景，目标是通过摄像头视频流对学生上网课时的状态进行分析，从而辅助判断学生是否处于正常学习状态。系统整体可以划分为视频采集与预处理、特征提取、状态估计、界面展示等模块。其中，我主要负责视频采集与预处理模块，重点工作包括视频流读取、图像帧处理、人脸区域信息整理，以及向后续特征提取模块提供统一格式的数据。

在第一次作业中，我将该项目的软件过程划分为项目计划、需求分析、概要设计、详细设计与编码实现、集成与测试、文档与配置管理、评审与提交等阶段。这一过程既参考了软件生命周期过程的基本思想，也结合了课程项目周期短、团队规模小、目标以可运行原型和文档提交为主的特点进行了剪裁。

随着生成式AI、大语言模型、AI代码助手、智能测试工具等技术的发展，软件工程过程正在从传统的人工主导模式逐渐转向“人机协同”的智能化模式。AI技术已经开始参与需求分析、系统设计、编码实现、测试验证、运维监控等多个环节。本次作业要求结合原有软件过程，分析AI技术可以在哪些环节提升效率和质量。因此，本报告将围绕“面向网课的智能监督系统”的实际过程，调研2024—2026年AI技术在软件工程中的应用趋势，并识别适合本项目的关键应用场景。

二、AI技术在软件工程中的应用趋势

2.1 需求分析阶段：从人工整理到AI辅助需求工程

在传统软件过程中，需求分析主要依赖人工访谈、会议讨论、文档整理和经验判断。对于课程项目来说，需求往往来源于老师给出的任务要求、小组成员的讨论以及对项目目标的理解。由于学生项目时间有限，需求分析文档容易出现功能描述笼统、边界不清楚、异常情况考虑不足等问题。

近年来，生成式AI开始应用于需求工程领域。大语言模型可以根据自然语言描述，辅助生成需求列表、用户故事、功能边界、非功能需求和验收标准。2024年关于生成式AI在需求工程中应用的系统综述指出，生成式AI在

需求抽取、需求分析、需求规格说明等任务上具有应用潜力，但也存在可解释性、可复现性和可控性不足等问题。因此，在实际软件过程中，AI更适合作为需求分析的辅助工具，而不是完全替代需求分析人员。

对于“面向网课的智能监督系统”而言，AI可以根据项目背景自动整理出系统应该具备的功能，例如摄像头视频流读取、图像帧处理、人脸区域提取、学生状态识别、结果展示和异常提示等。同时，AI还可以补充一些人工容易忽略的非功能需求，例如实时性、稳定性、隐私保护、异常输入处理等。

2.2 系统设计阶段：从人工绘图到AI辅助架构与接口设计

系统设计阶段需要将需求转化为系统结构、模块划分、接口说明和数据流关系。传统方式下，设计文档主要由成员手工完成，容易出现模块边界不清、接口字段不统一、流程图不规范等问题。对于小组课程项目来说，这类问题在后续集成阶段会更加明显。

AI技术可以在系统设计阶段发挥辅助作用。开发人员可以将需求说明输入AI工具，让AI生成初步的模块划分、数据流说明、接口字段表和流程图代码。例如，AI可以帮助生成Mermaid流程图，描述“摄像头输入—视频采集—预处理—特征提取—状态估计—界面展示”的整体流程。AI还可以检查不同模块之间的数据是否衔接，例如预处理模块输出的 `faces`、`owner_face_id`、`frame` 等字段是否能够满足后续特征提取模块的输入要求。

这种AI辅助设计方式适合本项目，因为本项目模块较多，但整体规模不大，不需要复杂的企业级架构设计。AI可以帮助提高设计文档的规范性和完整性，而人工成员仍然负责判断设计是否符合实际实现能力。

2.3 编码实现阶段：从手工编码到AI辅助开发

编码实现是AI技术当前应用较成熟的环节。以GitHub Copilot为代表的AI代码助手已经能够提供代码补全、代码生成、错误解释、测试生成、代码重构和Pull Request摘要等功能。GitHub Copilot官方文档中也明确列出了代码生成、调试错误、分析功能、生成测试、代码重构等典型使用场景。

对于本项目来说，AI可以辅助完成一些重复性较强、规则较明确的代码。例如，在视频采集与预处理模块中，AI可以帮助生成摄像头读取函数、空帧判断逻辑、数据封装函数、异常处理代码和日志输出代码。开发人员可以先给出函数说明、输入输出格式和异常处理要求，再让AI生成代码初稿。

但是，AI生成代码不能直接作为最终代码使用。原因在于，AI可能会生成不符合项目环境的库调用，也可能忽略本项目已有的接口约定。特别是涉及摄像头、图像处理和人脸检测的代码时，实际运行环境、依赖库版本和输入数据格式都会影响代码能否运行。因此，本项目中AI的定位应是“辅助生成和辅助审查”，最终代码必须由开发人员理解、修改、运行和确认。

2.4 测试验证阶段：从人工经验测试到AI辅助测试设计

测试验证是本项目中最适合引入AI技术的环节之一。传统课程项目中，测试往往主要关注正常流程是否能跑通，而对异常情况、边界情况和接口一致性的测试不足。例如，在“面向网课的智能监督系统”中，预处理模块不仅需要处理摄像头正常输入，还需要处理摄像头打不开、视频帧为空、画面中无人脸、画面中多人脸、人脸框越界、帧率过低等情况。如果这些异常场景没有被测试，系统在演示或集成时就可能出现问题。

AI可以根据需求文档、接口说明和模块功能描述生成测试用例表，自动补充正常场景、异常场景、边界场景和性能场景。2024—2025年的软件质量工程相关报告也显示，生成式AI正在成为测试设计、测试自动化和质量工程中的重要趋势。AI辅助测试的优势在于能够快速扩展测试场景，帮助开发人员发现人工容易忽略的问题。

在本项目中，可以选择“AI辅助测试用例生成”作为后续原型验证场景。原因是该场景输入明确、输出清晰，便于与传统人工方式进行对比，也便于使用测试用例数量、异常场景覆盖数、耗时等指标进行量化评价。

2.5 运维监控阶段：从手工排查到AI辅助日志分析

运维监控通常用于正式上线系统，包括异常检测、性能监控、根因分析、容量规划和自动扩容等内容。虽然本项目只是课程项目，不涉及长期线上运行，但在系统集成、课堂展示和最终验收阶段，仍然可能遇到运行异常。例如摄像头无法打开、视频画面卡顿、模块接口字段缺失、状态估计结果为空、界面无法显示等问题。

AI可以辅助分析日志和报错信息。开发人员可以将错误信息、运行日志和相关代码片段输入AI，让AI给出可能原因和排查步骤。例如，如果程序提示 `faces` 字段不存在，AI可能会建议检查预处理模块输出数据结构和特征提取模块输入字段是否一致。如果摄像头无法打开，AI可能会建议检查设备编号、权限设置、依赖库安装和运行环境。

因此，在本课程项目中，可以保留轻量级的AI辅助日志分析环节，不需要建立复杂的AIOps平台，只需要在调试过程中使用AI帮助分析错误即可。

三、原有软件过程存在的问题

结合第一次作业中定义的软件过程，本项目原有过程已经覆盖了项目计划、需求分析、概要设计、编码实现、集成测试、文档管理和最终提交等主要环节，整体上适合课程项目。但是，从AI技术应用和过程改进的角度看，原有过程仍然存在一些不足。

第一，需求分析阶段主要依赖人工整理，容易遗漏非功能需求和异常场景。例如，原有需求可能只强调系统能够识别学生状态，但没有充分说明摄像头异常、无人脸、多人人脸、低帧率等情况下系统应该如何处理。

第二，概要设计阶段的接口说明可能不够细致。本项目由多个模块组成，预处理模块向后续模块传递的数据字段非常关键。如果 `timestamp`、`faces`、`owner_face_id`、`frame` 等字段没有在文档中明确类型、含义和异常处理方式，后续集成时就容易出现理解偏差。

第三，编码实现阶段容易出现不同成员代码风格不统一、异常处理不完整、接口字段命名不一致等问题。课程项目时间有限，成员通常会优先实现功能，而不是系统地检查代码质量。

第四，测试验证阶段容易偏重正常流程测试，而缺少异常测试和边界测试。例如，只测试摄像头正常打开和单人人脸场景是不够的，还需要测试摄像头不可用、空帧、无人脸、多人脸、人脸框越界等情况。

第五，文档与配置管理阶段主要依靠人工维护，不同文档之间容易出现表述不一致。例如需求文档、概要设计文档、测试报告中对同一模块或字段的名称可能不完全一致。

因此，本次AI技术场景匹配的重点不是推翻原有过程，而是在原有过程的关键薄弱环节中加入AI辅助节点，提高过程效率和文档质量。

四、AI技术应用场景匹配分析

4.1 场景一：AI辅助需求整理与需求一致性检查

4.1.1 场景描述

在需求分析阶段，可以引入生成式AI辅助整理需求。项目成员先将项目背景、用户场景、课程要求和已有想法输入AI工具，由AI生成结构化需求清单，包括功能需求、非功能需求、数据需求、异常场景和验收标准。

对于本项目，AI可以辅助生成以下需求内容：

- 系统应能够读取摄像头视频流；
- 系统应能够对视频帧进行基础预处理；
- 系统应能够识别或接收人脸区域信息；
- 系统应能够输出统一格式的数据结构；
- 系统应能够处理无人脸、多人脸、空帧、摄像头异常等情况；
- 系统应具有一定实时性和稳定性；
- 系统应避免直接保存或泄露用户隐私数据。

4.1.2 与原有过程的匹配

该场景对应原有软件过程中的“需求分析过程”。原有需求分析依靠人工讨论和文档编写，引入AI后，可以先由AI生成需求初稿，再由小组成员进行筛选、修改和确认。

4.1.3 预期改进效果

改进维度	原有方式	AI辅助方式	预期效果
需求整理效率	人工逐条编写	AI生成初稿后人工修改	初稿编写时间减少
需求完整性	容易遗漏异常场景	AI补充异常和非功能需求	覆盖范围更全面
需求一致性	不同成员表述不统一	AI辅助统一术语	文档更规范
可验证性	验收标准不清楚	AI生成验收标准建议	便于后续测试

4.1.4 注意事项

AI生成的需求不能全部采纳。对于超出课程范围、实现难度过高或与项目目标不一致的需求，应由人工删除。例如，AI可能提出复杂的学习行为预测模型或云端监控平台，这些功能对于课程项目来说可能过重，不适合作

为本次实现目标。

4.2 场景二：AI辅助概要设计与接口说明生成

4.2.1 场景描述

在概要设计阶段，可以使用AI辅助生成模块结构、数据流说明、接口字段表和流程图。本项目中，视频采集与预处理模块需要向后续特征提取模块传递统一数据，因此接口设计非常关键。

AI可以根据模块说明生成如下接口表：

字段名	类型	含义	异常情况
timestamp	float	当前帧时间戳	若无法读取则记录异常
frame	ndarray	当前图像帧	空帧时返回错误状态
faces	list	人脸检测结果列表	无人脸时为空列表
face_id	int / str	人脸编号	无人脸时为空
face_roi	tuple / list	人脸区域坐标	越界时需要修正或丢弃
owner_face_id	int / str / None	主体人脸编号	无法确定时为None

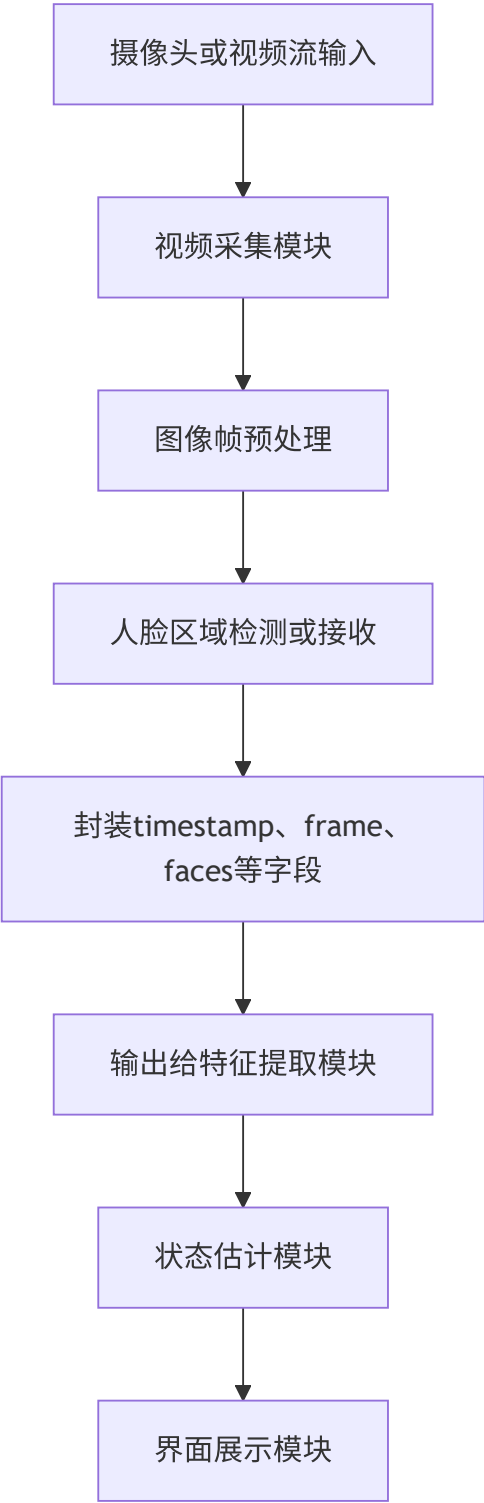
4.2.2 与原有过程的匹配

该场景对应原有软件过程中的“概要设计过程”。原有过程已经要求明确模块功能、输入输出、处理逻辑、接口和异常处理方式。AI可以帮助生成初稿和检查遗漏，但最终接口仍需由负责不同模块的成员共同确认。

4.2.3 预期改进效果

改进维度	原有方式	AI辅助方式	预期效果
接口说明	人工编写，容易遗漏	AI生成字段表	字段更完整
流程图	手工绘制	AI生成Mermaid代码	文档可视化更方便
异常处理	容易被忽略	AI主动补充异常情况	后续联调风险降低
模块边界	依赖成员理解	AI辅助检查输入输出	边界更清晰

4.2.4 示例流程图



4.3 场景三：AI辅助编码实现与代码审查

4.3.1 场景描述

在详细设计与编码实现阶段，可以使用AI代码助手辅助生成基础代码和进行代码审查。对于视频采集与预处理模块，AI可以辅助完成以下任务：

- 生成摄像头读取函数；
- 生成视频帧预处理函数；
- 生成输出数据结构封装函数；
- 生成异常处理逻辑；
- 解释代码运行错误；
- 检查代码命名和字段是否一致；
- 生成简单单元测试代码。

4.3.2 与原有过程的匹配

该场景对应原有软件过程中的“详细设计与编码实现过程”。原有过程要求成员按照分工完成模块代码，并保证与其他模块能够正确对接。AI可以提高基础代码编写效率，但不能替代开发人员对代码逻辑的理解。

4.3.3 预期改进效果

改进维度	原有方式	AI辅助方式	预期效果
代码编写	人工从零编写	AI生成代码框架	提高编码效率
错误排查	人工搜索和尝试	AI解释报错原因	缩短调试时间
代码规范	依赖个人习惯	AI辅助审查	命名和结构更统一
测试代码	人工较少编写	AI生成测试样例	提高测试意识

4.3.4 注意事项

AI生成代码必须经过人工检查和本地运行验证。尤其是涉及摄像头、OpenCV、图像数组、人脸检测模型等内容时，AI可能生成不符合本地环境或项目接口的代码。因此，本项目中应明确规定：AI生成代码只能作为初稿，开发人员必须能够解释并修改代码后才能提交。

4.4 场景四：AI辅助测试用例生成

4.4.1 场景描述

测试验证阶段是本项目最适合引入AI的环节之一。开发人员可以将需求文档、接口说明和模块功能输入AI工具，让AI生成测试用例表。测试用例应覆盖正常场景、异常场景、边界场景和性能场景。

对于视频采集与预处理模块，AI可以辅助生成以下测试场景：

测试类别	示例场景
正常场景	摄像头正常打开，画面中存在单人脸
边界场景	画面中出现多人脸，人脸位于图像边缘
异常场景	摄像头无法打开，视频帧为空
数据格式场景	faces 为空， face_roi 格式错误
性能场景	连续处理多帧时帧率过低

4.4.2 与原有过程的匹配

该场景对应原有软件过程中的“集成与测试过程”。原有过程强调单个模块完成后，需要进行接口联调和系统测试。AI辅助测试可以补充人工容易忽略的异常情况，提高测试用例覆盖度。

4.4.3 预期改进效果

改进维度	原有方式	AI辅助方式	预期效果
测试用例数量	人工设计较少	AI快速生成更多用例	用例数量增加
异常覆盖	容易遗漏	AI主动补充异常场景	覆盖更全面
编写效率	人工逐条编写	AI生成初稿	时间减少
测试规范	格式可能不统一	AI生成表格化用例	文档更规范

4.4.4 适合作为原型验证场景的原因

本次作业后续需要选择一个核心改进场景进行最小可行验证。相比需求分析、概要设计和编码实现，AI辅助测试用例生成更容易验证效果。因为它可以通过测试用例数量、异常场景数量、生成耗时、人工修改比例等指标进行对比。因此，本文建议将“AI辅助测试用例生成”作为后续验证报告的核心场景。

4.5 场景五：AI辅助文档维护与一致性检查

4.5.1 场景描述

课程项目通常需要提交多份文档，包括项目计划、需求分析、概要设计、测试报告、周报和最终总结。由于文档数量较多，且不同成员负责不同部分，容易出现术语不统一、接口字段不一致、前后描述冲突等问题。

AI可以作为文档维护助手，帮助检查不同文档之间的内容一致性。例如，可以让AI检查需求文档和概要设计文档中的模块名称是否一致，检查 timestamp 、 faces 、 owner_face_id 、 frame 等字段在不同文档中的含义是否一

致，也可以让AI根据每周工作记录生成周报初稿。

4.5.2 与原有过程的匹配

该场景对应原有软件过程中的“文档与配置管理过程”和“评审与提交过程”。原有过程已经强调项目推进过程中需要持续维护文档，AI可以帮助提高文档质量，但最终文档内容必须由人工确认。

4.5.3 预期改进效果

改进维度	原有方式	AI辅助方式	预期效果
文档一致性	人工检查，容易遗漏	AI辅助比对字段和术语	减少前后矛盾
语言表达	表述可能不规范	AI辅助润色	提高可读性
周报总结	人工重复编写	AI生成初稿	减少重复劳动
最终整合	手动整合较慢	AI辅助总结结构	提高提交效率

4.5.4 注意事项

AI辅助文档维护时，不能让AI编造项目进度或夸大实际成果。所有文档内容必须与项目真实完成情况一致。尤其是测试结果、性能数据和功能完成情况，必须来源于实际记录或可复现实验，不能直接采用AI虚构数据。

五、关键应用场景优先级排序

结合本项目实际情况、课程项目规模、落地难度和预期收益，本文将AI技术应用场景优先级排序如下：

优先级	应用场景	对应软件过程	落地难度	预期收益
高	AI辅助测试用例生成	集成与测试过程	低	提高测试覆盖度，减少异常遗漏
高	AI辅助需求整理与一致性检查	需求分析过程	低	提高需求完整性和文档效率
中高	AI辅助概要设计与接口说明	概要设计过程	中	提高接口清晰度，降低联调成本
中	AI辅助编码实现与代码审查	编码实现过程	中	提高编码效率，减少低级错误
中	AI辅助文档维护与一致性检查	文档与配置管理过程	低	提高文档规范性和一致性

其中，“AI辅助测试用例生成”最适合作为本次作业后续的原型验证场景。原因是该场景输入明确、输出清晰，且能够用具体数据进行对比。例如，可以比较人工方式与AI辅助方式在测试用例数量、异常场景覆盖数量、耗时

等方面的差异。

六、AI技术引入的风险与边界

虽然AI可以在多个软件过程环节中提升效率，但也会带来一定风险。因此，在本项目中引入AI时，需要明确AI的使用边界。

6.1 AI幻觉风险

AI可能生成看似合理但实际上不存在或不适合本项目的内容。例如，它可能建议使用复杂的云端监控平台、自动扩容系统或大型深度学习训练流程，但这些内容并不适合课程项目。

应对方式是：所有AI输出必须经过人工审核，只有符合项目目标、团队能力和课程周期的内容才能被采纳。

6.2 数据安全与隐私风险

本项目涉及摄像头视频流和人脸区域信息。如果将真实视频帧、人脸图像或个人隐私数据上传到在线AI工具，可能造成隐私风险。

应对方式是：使用AI时只输入抽象后的模块说明、接口字段和错误信息，不上传真实人脸图像和隐私数据。

6.3 技术依赖风险

如果成员过度依赖AI生成代码和文档，可能导致自己并不理解项目实现逻辑，后续答辩、调试和维护都会受到影响。

应对方式是：AI生成内容只能作为初稿，项目成员必须能够解释自己负责模块的设计、代码和测试过程。

6.4 输出质量不稳定风险

AI生成的内容质量与提示词、上下文和工具能力有关，可能出现格式不统一、测试用例重复、代码不可运行等问题。

应对方式是：建立AI输出物质量标准，对需求、设计、代码、测试用例和文档分别进行人工检查。

七、结论

通过对2024—2026年AI技术在软件工程中的应用趋势进行调研，并结合“面向网课的智能监督系统”的原有软件过程，可以看出，AI技术能够在需求分析、系统设计、编码实现、测试验证和文档管理等多个环节发挥作用。

对于本项目而言，AI最适合承担辅助角色，而不是替代项目成员完成全部工作。在需求分析阶段，AI可以帮助整理需求、补充非功能需求和异常场景；在概要设计阶段，AI可以帮助生成接口表和流程图；在编码实现阶段，AI可以辅助生成代码框架和进行代码审查；在测试验证阶段，AI可以生成更完整的测试用例；在文档管理阶段，AI可以帮助检查术语和字段一致性。

综合落地难度和预期收益，本文认为“AI辅助测试用例生成”是最适合作为后续原型验证的场景。该场景能够直接服务于我负责的视频采集与预处理模块，也便于通过耗时、用例数量、异常覆盖度等指标进行量化对比。整体来看，在原有课程项目软件过程基础上引入AI辅助节点，可以提高过程效率、增强文档规范性、提升测试覆盖度，并使软件过程更加适应智能化开发趋势。

八、参考资料

[1] Cheng, H., Husen, J. H., Lu, Y., Racharak, T., Yoshioka, N., Ubayashi, N., & Washizaki, H. *Generative AI for Requirements Engineering: A Systematic Literature Review*. arXiv, 2024.

<https://arxiv.org/abs/2409.06741>

[2] GitHub Docs. *GitHub Copilot features*. GitHub, 2024-2026.

<https://docs.github.com/en/copilot/get-started/features>

[3] GitHub Docs. *Using GitHub Copilot code review*. GitHub, 2024-2026.

<https://docs.github.com/copilot/using-github-copilot/code-review/using-copilot-code-review>

[4] Google Cloud DORA. *Accelerate State of DevOps Report 2024*. Google Cloud, 2024.

<https://dora.dev/research/2024/dora-report/>

[5] Capgemini, Sogeti, OpenText. *World Quality Report 2024-25*. Capgemini Research Institute, 2024.

<https://www.capgemini.com/insights/research-library/world-quality-report-2024-25/>

[6] Sengar, S. S., Hasan, A. B., Kumar, S., & Carroll, F. *Generative Artificial Intelligence: A Systematic Review and Applications*. arXiv, 2024.

<https://arxiv.org/abs/2405.11029>

[7] 中国信息通信研究院. 《智能化软件工程技术和应用要求 第3部分：智能测试能力》，2024.

[8] Gtest全球软件测试技术峰会. 2025年会议资料与智能测试相关公开资料, 2025.